

EGO BLOCKCHAIN

Decentralized 5G-Powered Blockchain

Artit Muhaxhiri

September 30, 2025

www.egoblockchain.com

Abstract

You might question the need to introduce another layer one blockchain and add another cryptocurrency to a market already crowded with 22,932 options. However, each blockchain has unique features and capabilities that can address specific challenges and demands, and the introduction of new blockchains and cryptocurrencies can pave the way for innovation and new possibilities in the industry.

Our proposed solution aims to address two major problems in the industry: data storage and cryptocurrencies. The current centralized data storage systems are vulnerable to hacking and data breaches, which can compromise sensitive information. On the other hand, Bitcoin has faced criticism for its energy-intensive Proof of Work (PoW) mechanisms and the potential centralization of infrastructure. PoW mechanisms used in Bitcoin result in wasted energy spent validating proofs, which present no other utility. With PoW, miners benefit from economies of scale, which can lead to infrastructure centralization and work against the goal of decentralization. With Proof of Stake (PoS), centralization of infrastructure can occur out of convenience, as stakers often run their nodes on centralized cloud infrastructure, weakening the network's resilience to attacks.

To address these issues, we are creating a decentralized storage system that utilizes blockchain and Ego 5G devices to ensure data security and privacy. Micro-servers will be spread worldwide to allow individuals and companies to store their data in an encrypted manner. We introduce Deterministic Reward Scoring (DRS) — a transparent, reproducible scoring system that evaluates node performance (uptime, storage proofs, coverage quality, serve ratio) and adjusts rewards fairly, ensuring resilience without centralization.

Our proposed solution addresses the criticisms of blockchains and cloud services while offering improved security, a more robust ecosystem, reduced costs, passive income for storage providers, and enhanced privacy features.

Introduction

Many people think that Web3 is a completely new technology that operates independently of Web1 and Web2. The truth is that Web3 is built on top of the existing infrastructure provided by Web1 and Web2. This means that even if the dApp's functionality is decentralized, the frontend or user interface can still be removed or deleted, which would make the dApp inaccessible to users.

Decentralized networks have been gaining momentum in recent years as a way to provide more security, privacy, and control over data. However, the transition from Web2 to Web3 has been slow

due to the lack of sufficient decentralized infrastructure, especially outside the realm of DeFi.

The setup of decentralized nodes is often complicated, time-consuming, and costly, which can discourage developers and end-users from adopting decentralized applications.

We are developing a new blockchain with a main focus on decentralized storage and smart contract functionality to solve these issues. The blockchain will utilize a proof-of-spacetime and proof-of-coverage consensus mechanism to ensure security and reliability.

With new 5G micro servers to provide fast and reliable infrastructure for decentralized applications. We will enable users to interact with dApps seamlessly, without experiencing lag or delays. Furthermore, we will work in conjunction with IPFS, a protocol for distributed storage and sharing of files. This will enable developers to create decentralized applications that rely on decentralized storage, ensuring data redundancy and increasing fault tolerance. By leveraging decentralized storage, smart contracts, and fast network infrastructure, Ego Blockchain aims to solve the issue of removing the frontend of a dApp. This will help to create a more robust and scalable decentralized network that can encourage the development and adoption of decentralized applications beyond DeFi. With the elimination of infrastructure costs for developers and end-users, we hope to encourage the creation of more open-source dApps, leading to a diverse set of applications and mainstream adoption of the decentralized web. The potential benefits of decentralized networks are clear, including increased security, privacy, and control over data. We aim to provide a viable solution to the challenges that have slowed the adoption of Web3, making decentralized applications more accessible and user-friendly.

1. Decentralized Storage and 5G Wireless Network

This whitepaper outlines the architecture of a decentralized storage and wireless network powered by micro 5G servers functioning as edge nodes. Data undergoes client-side encryption, is partitioned into shards, and is replicated across diverse nodes. Continuous integrity is maintained through Proof-of-Replication (PoRep) and Proof-of-Spacetime (PoSt) mechanisms. Regional rollups aggregate transactions and proof digests off-chain, posting compact commitments with data-availability manifests to layer-1 (L1) shards, where a challenge window allows for dispute resolution. A transparent Deterministic Reward Scoring (DRS) mechanism governs rewards, ensuring fairness without reliance on artificial intelligence.

Core Components and Definitions

The system comprises a network of micro 5G server nodes, content shards, clients, L1 consensus shards, and regional rollup operators. Micro 5G servers act as edge nodes, hosting data and participating in consensus. Content is divided into encrypted shards for efficient storage and resilience. Clients initiate data uploads and retrievals, while consensus shards manage accounts, deals, and execution states. Regional rollup operators batch transactions for scalability. Replication ensures data durability, typically maintaining three copies across distinct nodes to mitigate risks.

Content Sharding and Identification

To enable efficient storage and enhance resilience, content is divided into smaller, manageable pieces known as shards. Each piece is encrypted on the client side before being uploaded to the network, ensuring privacy from the outset. A unique identifier for each shard is generated by hashing the content and mapping it to a predefined number of shards. This identifier is computed using the formula $\text{shard_id_content} = \text{hash}(\text{piece}) \% m$, where $\text{hash}(\text{piece})$ is a cryptographic hash of the content piece and m represents the total number of shards in the partitioning scheme. The system typically maintains a replication factor of three, meaning each shard is stored on three distinct nodes. To prevent correlated failures, placement adheres to diversity constraints, ensuring shards are distributed across different operators and geographic regions.

Node Placement Strategy

Data placement employs a deterministic mapping, where a collision-resistant hash of a unique identifier, such as a content ID, determines the node index. This process is repeated for each replica, ensuring diverse placement to avoid single points of failure. The approach balances simplicity with robustness, safeguarding data across the network.

Encryption and Content Addressing

Clients encrypt content using a symmetric authenticated encryption with additional data (AEAD) scheme. Optionally, per-provider key wrapping with a key encapsulation mechanism (KEM) ensures providers never access plaintext. Content is addressed via a content identifier (CID), computed as a hash of the ciphertext. Commitments, such as Merkle roots over content or replica sets, bind data to specific storage replicas, ensuring verifiability.

Data Retrieval Process

Clients request data by specifying shard identifiers. Nodes holding these shards respond with encrypted chunks and inclusion proofs. The client verifies these proofs against on-chain Merkle roots and decrypts the data locally, ensuring privacy and integrity throughout the process.

Consensus Sharding and Cross-Shard Communication

The system employs consensus sharding to manage accounts, deals, and execution states. Shard assignments are determined by hashing relevant identifiers, such as account or deal IDs. Cross-shard communication relies on receipts, which include Merkle proofs tied to the source shard's receipts root. These proofs are verified by the destination shard before processing, ensuring secure and reliable interactions.

Proof Mechanisms

Proof-of-Replication (PoRep) ensures that each node seals a unique, replica-bound encoding of its data copy, preventing outsourcing or on-demand generation attacks. Proof-of-Spacetime (PoSt) verifies continuous data custody through randomized audits based on finalized block randomness. Successful PoSt responses earn rewards, while failures trigger penalties, repairs, or slashing for fraud. Proof events are anchored in shard headers, with full evidence bundles served off-chain via content identifiers.

Regional Rollups and Data Availability

Regional rollups play a critical role in scaling the network by batching transactions and proof digests off-chain, reducing the load on layer-1 (L1) shards. For a rollup operator in a given epoch, a batch is formed containing transactions, proof event digests, and large data payloads. Commitments are computed to anchor this batch on-chain, including a transaction root as $tx_root = \text{Merkle}(Tx_batch)$, a proof digest root as $proofs_root = \text{Merkle}(digests\ of\ PoC,\ PoSt,\ PoRep\ events)$, and a data-availability manifest root as $da_root = \text{Merkle}(manifest\ describing\ erasure-coded\ blobs)$. Large payloads, such as state snapshots or user files, are erasure-coded into k data chunks and m parity chunks, resulting in a total of $n = k + m$ chunks. During a challenge window, light

clients and challengers sample s chunks to verify availability. The probability of detecting f missing chunks is given by the formula $P_{\text{detect}} = 1 - ((n - f)/n)^s$, providing a high likelihood of identifying data unavailability. Operators must serve requested chunks within a response window, and failures or invalid responses can trigger disputes, potentially leading to slashing of the operator's stake.

Deterministic Reward Scoring

The Deterministic Reward Scoring (DRS) mechanism transparently calculates rewards based on on-chain evidence. Factors include node uptime, PoSt pass rates, latency, retrieval performance, and penalties for faults or fraud. Scores are weighted and mapped to a bounded reward multiplier, ensuring predictable and fair payouts. Slashing remains evidence-based, maintaining trust in the system.

Shard Header Commitments

Shard headers anchor the system's integrity through compact cryptographic roots. These include transaction roots, state roots, receipt roots, storage proof event roots, and data-availability manifests. All fetched data or proofs can be verified against these anchors, ensuring consistency and trustworthiness.

Operational Workflow

The process begins with clients encrypting and sharding data, computing content identifiers, and selecting nodes for replication with diversity constraints. Selected nodes seal replicas using PoRep, anchoring proof digests on-chain while publishing full evidence off-chain. Nodes respond to periodic PoSt challenges, with results batched by rollups and committed on-chain. Large payloads, such as state snapshots or user files, are erasure-coded and referenced in rollup commitments. Clients retrieve data by requesting shard identifiers, verifying proofs, and decrypting locally.

Compliance and Deployment Considerations

Optional coverage attestations leverage passive cellular measurements with side-channel nonces to bind time and identity. Data privacy is preserved through client-side encryption and regional residency-aware placement. Heavy payloads favor wired or Wi-Fi connectivity, while lightweight meta-events can utilize cellular links. The system is designed for flexibility, supporting diverse deployment scenarios while maintaining cryptographic accountability.

System Rationale

This architecture delivers integrity through on-chain commitments, availability via data sampling and custody audits, and scalability through regional rollups. Validators focus on consensus-critical data, while storage providers and rollup operators handle intensive off-chain tasks under cryptographic oversight. The transparent DRS mechanism ensures fair incentives, fostering a robust and decentralized ecosystem.

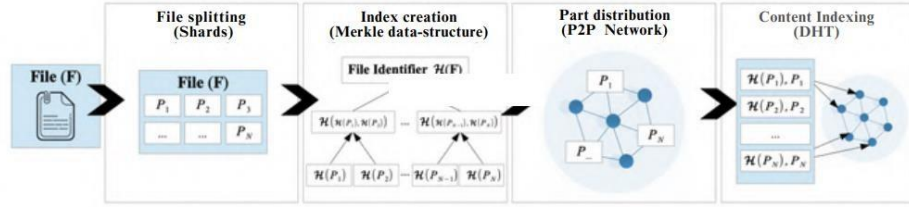


Figure 1. Block scheme of the decentralized storage protocol for splitting a file F into N pieces P_n stored along with the hash $H(P_n)$ in the peers.

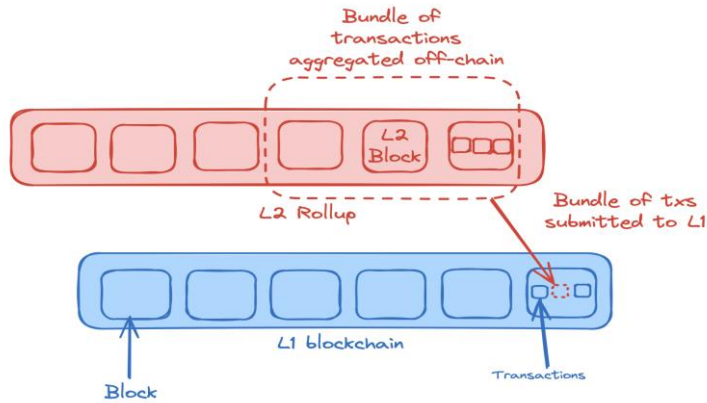


Figure 1.1. Block scheme illustrating the aggregation and submission of **Layer 2 (L2) Rollup** transaction bundles to the **Layer 1 (L1) blockchain**.

2. Quantum-Resistant Key Pairs (PQC-first for Ego)

Why PQC Quantum computers threaten legacy crypto (RSA/ECC) via Shor's algorithm. Ego adopts NIST-standard post-quantum cryptography built on hard problems believed secure against both classical and quantum attacks.

Algorithms we use

- ML-KEM (Kyber-768, FIPS 203): Key encapsulation for shared secrets, per-session/per-output keys.
- ML-DSA (Dilithium-2, FIPS 204): Digital signatures for transactions, blocks, PoC/PoS attestations, rollup commits, and node identity.
- SLH-DSA (SPHINCS+): Anchors only (e.g., OTA releases, epoch/DRS checkpoints).
- Symmetric AEAD: XChaCha20-Poly1305 for authenticated encryption.
- Hash/KDF: BLAKE2s-256 and HKDF-BLAKE2s with strict domain separation.

Where keys are used

- Node identity and governance: Dilithium-2 (dual-sign with Ed25519 only during the transition).
- Transactions and blocks: Dilithium-2 signatures on all txs and block headers.
- P2P streams: Hybrid handshake X25519 + Kyber $\rightarrow$ HKDF-BLAKE2s $\rightarrow$ XChaCha20-Poly1305 (PQ-only later).
- Storage/PoC/PoS evidence: Dilithium-2 signatures on all events and bundles (on-chain digests, off-chain CIDs).
- OTA releases: Threshold-signed artifacts with Dilithium-2 (Ed25519 verify-only during transition).

Keys at rest and addressing

- Signing keys (Dilithium-2): generated and stored in TPM/SE on devices; non-exportable. Address = `versioned_blake2s(dilithium_pk)[0..20]`.
- KEM keys (Kyber-768): for content keys and stream handshakes; device-scoped or ephemeral.
- All signed records include `alg_sig_id` and network metadata to prevent replay across networks.

Migration policy

- Hybrid phase: dual signatures (Ed25519 + Dilithium); hybrid sessions (X25519 + Kyber).
- PQ-required phase: consensus/proofs topics require PQ-capable peers; downgrade attempts rejected.
- PQ-only phase: on-chain signatures must be Dilithium; Ed25519 retained only for legacy verification tools.

Sizes (approximate)

- Kyber-768: pk 1184 B, ciphertext 1088 B
- Dilithium-2: pk 1312 B, sk 2528 B, signature $\approx$ 2420 B
- Ed25519 (transition): pk 32 B, sig 64 B
- X25519 (transition): pk 32 B
- SPHINCS+ (anchors only): signature 8–17 KB

Deterministic hashing (all signatures)

- Canonical encode (CBOR/Protobuf, deterministic).
- Domain-separated digests: $H = \text{blake2s}(\text{"ego/ctx:"} \parallel \text{context} \parallel \text{bytes})$
- Examples:
 - `"ego/core/v1"` (BlockHeaderCore);
 - `"ego/tx/v1"` (transactions);
 - `"ego/event/post/v1"` (PoSt);
 - `"ego/peerbind/v1"` (P2P binding).

Example: Hybrid stream handshake (P2P)

- Identify: peer advertises supported PQC, public keys, and signs the announcement with Dilithium (plus Ed25519 during transition).
- Handshake:
 - Client encapsulates to server's Kyber pk $\rightarrow$ `ss_pq`.
 - Optionally computes X25519 shared secret $\rightarrow$ `ss_x`.
 - $\text{IKM} = \text{ss_pq} \parallel \text{ss_x}$; derive `k_tx/k_rx` via HKDF-BLAKE2s (domain-tagged).
 - Encrypt frames with XChaCha20-Poly1305 (unique 24-byte nonces; AAD binds stream id/alg ids).

Example: Transaction with stealth output (optional privacy mode)

- Sender encapsulates to receiver's Kyber pk $\rightarrow$ shared secret.
- Derives a one-time signing key (OTS) deterministically from the secret.
- Encrypts output metadata (amount, memo) with XChaCha20-Poly1305.
- Publishes the one-time Dilithium public key in TxOut; receiver scans with their Kyber sk to recognize and spend by signing with the OTS key.

| Ego Key Generation |

|



| Kyber-768 KEM keypair |
| (pk_kem, sk_kem) |

|



| Dilithium-2 signature keypair |
| (pk_sig, sk_sig) |

|



| Hybrid phase (transition) |
| Add Ed25519/X25519 keys |

|



| Publish public data: |
| {pk_kem, pk_sig, (pk_ed?)} |
| + algorithm/version fields |

▼
Use cases:
- Kyber: encaps/decaps →
derive shared secrets
- Dilithium: sign tx/blocks
and PoC/PoS proofs
- Ed25519/X25519: legacy
only during transition
▼
PQ-only cutover:
Require Dilithium + Kyber
(drop legacy keys)

3. Encryption with ML-KEM (Kyber-768) and XChaCha20-Poly1305

Overview

- Confidentiality: encapsulate to the receiver's ML-KEM (Kyber-768) public key to get a shared secret, derive an AEAD key, and encrypt the payload with XChaCha20-Poly1305 using a unique 24-byte nonce.
- Authenticity: sign a domain-separated digest of the envelope with ML-DSA (Dilithium-2).

Key and artifact sizes (how they look)

- ML-KEM public key (pk_mlkem): 1184 bytes (example, base64-truncated) pk_mlkem: "m6y1g1p2w4yGQw7c...AA=="
- ML-KEM secret key (sk_mlkem): 2400 bytes (kept secret; base64-truncated) sk_mlkem: "tq9R0WZrJfP1a7rZ...AA=="
- ML-KEM ciphertext (ct_kem): 1088 bytes per message (base64-truncated) ct_kem: "T1nZt2Hk5m0bq2rA...AA=="

- AEAD nonce (24 bytes): hex (example) nonce24:
"1ae10b594f09e26a7e902ecbd060069175a9b1d4a7f1b2c3"
- AEAD ciphertext (variable; plaintext + 16-byte tag): hex-truncated ciphertext:
"a22c43d12bb38bdf7b526eabc34b4860c8dc9de594..."

Envelope format (deterministic; signed) EncryptedMessage { version: 1, alg_kem: "ML-KEM-768", alg_aead: "XChaCha20-Poly1305", receiver_pk_id: blake2s(receiver_pk_mlkm), // or carry full pk ct_kem: bytes(1088), nonce24: bytes(24), aad: { chain_id, sender_addr, tx_id, alg_ids, ... }, // authenticated, not encrypted ciphertext: bytes, // XChaCha20-Poly1305(plaintext, aad) alg_sig: "ML-DSA-2", sig_dilithium: bytes // Dilithium-2 signature by sender }

Send flow (encrypt + sign)

1. Encapsulate
 - a. (ct_kem, ss) = ML-KEM.Encaps(receiver_pk_mlkm)
2. Derive AEAD key
 - a. K = HKDF-BLAKE2s(ss, info="ego/msg/v1")
3. Encrypt
 - a. nonce24 = 24 random bytes (unique per K)
 - b. ciphertext = XChaCha20-Poly1305.Seal(K, nonce24, aad, plaintext)
4. Sign envelope
 - a. H = blake2s("ego/ctx:env" || canonical(envelope_without_sig))
 - b. sig_dilithium = Dilithium.Sign(H, sender_sk_dilithium)
5. Transmit EncryptedMessage

Receive flow (verify + decrypt)

1. Verify signature
 - a. H = blake2s("ego/ctx:env" || envelope_without_sig)
 - b. Dilithium.Verify(H, sig_dilithium, sender_pk_dilithium)
2. Decapsulate and derive key
 - a. ss = ML-KEM.Decaps(ct_kem, receiver_sk_mlkm)
 - b. K = HKDF-BLAKE2s(ss, info="ego/msg/v1")
3. Decrypt
 - a. plaintext = XChaCha20-Poly1305.Open(K, nonce24, aad, ciphertext)

Worked mini-example (illustrative)

- Plaintext: "This is a secret message"
- aad: {chain_id: 1, sender: 0xAB..., alg: ML-KEM-768|XChaCha20}
- ct_kem: "T1nZ...AA==" (1088 B)
- nonce24: "1ae10b594f09e26a7e902ecbd060069175a9b1d4a7f1b2c3"
- ciphertext (truncated): "a22c43d12bb38bdf7b526eabc34b4860c8dc9de594..."
- sig_dilithium: "qVwL...AA==" (≈2420 B)

Rules and notes

- Use unique 24-byte nonces per AEAD key; never reuse (key, nonce).
- Always include strong AAD (addresses, chain_id, tx/stream id, alg ids) so integrity binds to context.
- All signatures use domain-separated, canonical encodings (e.g., "ego/tx/v1", "ego/ctx:env").

The whitepaper is a living document; implementation details may evolve. Track progress at <https://github.com/egoblockchain>.

4. Ego Protocol

Transport layer (P2P mesh)

- Protocols: libp2p over QUIC (UDP) with TCP fallback, NAT traversal (DCUtR), IPv6.
- Discovery/routing: Kademlia DHT (providers + signed values), dnsaddr; mDNS dev/LAN only.
- Gossip: GossipSub v1.2 topics per shard (tx, headers, receipts, proofs), plus rollup/disputes and PoC H3 cells.
- Admission (Ego-only): Identify extension carries a Dilithium device certificate + TPM/SE attestation; protected topics require valid attestation.
- Resource control: per-peer/per-topic caps, backpressure; cellular-safe mode caps peers and disables heavy streams on 5G.

Consensus layer (sharded BFT)

- Model: HotStuff/Tendermint-style BFT per shard ($n = 3f+1$, quorum $2f+1$), micro-slot ≈ 0.1 s; epoch aggregation targets 1–3 s global finality.
- Header commitments: tx_root, state_root, receipts_root, events_root_post (PoSt/PoRep), events_root_poc (PoC), rollup_root, da_root; proposer signs with Dilithium-2.
- Quorum certificates: votes signed with Dilithium-2; QC = bitmap + sigs_root (Merkle root over votes).
- Randomness: epoch/window seeds from finalized headers (vrf_output); bind PoC/PoSt challenges to (deal_id, replica_id, epoch).

Proof-of-Coverage (PoC)

Purpose

- Prove that a device provided real wireless coverage at a given time and place.
- Produce a public, auditable record (PoCEvent) with a quality score $q \in [0,1]$ used by DRS.

Signals and metrics (UE-compliant)

- Cellular (passive): LTE/NR downlink metrics captured by the UE:
 - RSRP (signal power), RSRQ, SINR, PCI, NR-ARFCN, optional SSB/beam index, serving/neighbor flags
 - Optional Timing Advance (if modem exposes it)
- Location/time: GNSS coordinates (lat/lon/alt), DOP/quality, UTC time
- Side-channel binding (unlicensed): BLE/Wi-Fi co-beacon carries a signed $\{\text{nonce16}, \text{time32}, \text{beacon_id}\}$ to bind witness measurements to the window, resist SDR replay

Challenge and scheduling

- Challenge windows: short (e.g., $W=10$ s), scheduled by epoch randomness
 - $R_e = H(\text{vrf_output} \parallel \text{region_id} \parallel \text{epoch} \parallel \text{slot})$
 - Select a device per H3 cell; announce window_start/end, NR band info, and nonce
- Announcement: BeaconAnnouncement $\{\text{beacon_id}, \text{h3_cell}, \text{window_start/end}, \text{nr_arfcn/band}, \text{nonce16}\}$ signed with Dilithium-2; start BLE/Wi-Fi co-beacon

Witnessing (nearby nodes)

- During the window, witnesses:
 - Capture cellular metrics (RSRP/RSRQ/SINR/PCI/ARFCN), GNSS fix/time/DOP
 - Record co-beacon (nonce, signed by beacon)
 - Build WitnessReport (Dilithium-signed)
 - Rate-limit and batch (cellular-safe: 0.5–1 Hz scanning, 5–10 s batching)

Aggregation and verification

- Aggregator collects announcements and reports, then:
 - Verifies signatures (Dilithium), window timing bounds
 - Ensures ≥ 3 independent witnesses from ≥ 2 H3 cells and ≥ 2 accounts
 - Co-beacon binding: fraction of witnesses saw valid nonce
 - Path-loss/geometry: fit to 3GPP TR 38.901 UMa/UMi
 - Use distance vs RSRP with acceptable RMSE τ (e.g., 6–10 dB urban)
 - Penalize impossible combos (e.g., high RSRP + very poor SINR)
 - Diversity and radius spread: multiple distinct H3 cells and radial distribution
 - Compute deterministic quality score: $q = 0.4 \cdot \text{fit} + 0.2 \cdot \text{diversity} + 0.2 \cdot \text{radius} + 0.2 \cdot \text{nonce} - \text{penalties}$
Clamp $q \in [0,1]$; accept if $q \geq q_{\min}$ (e.g., 0.5)
 - Density penalty (anti-clustering):
 - For n devices time-weighted co-located in $\sim 1 \text{ m}^2$ (dominant microcell over the epoch): $\text{LDM} = \max(1 - 0.10 \cdot (n-1), 0.40)$
 - Apply to PoC reward components and proposer utility bonus (PUC)
 - Publish DensityEvent $\{\text{node_id}, \text{h3_cell}, n, \text{LDM}, \text{evidence_root}\}$
 - Emit PoCEvent on-chain under events_root_poc: $\{\text{region_id}, \text{epoch}, \text{beacon_hash}, \text{witnesses_root}, \text{agg_digest}, \text{poc_quality_fp16}, \text{cid_hint}, \text{aggregator_sig}\}$ Evidence bundles (beacon/witness reports, coherence stats) stored off-chain by CID; anchor daily roots

Anti-gaming and integrity

- Replay: unique (beacon_id, nonce, epoch); co-beacon signed; stale/duplicate nonces rejected
- Self-witness loops: down-weight or reject clusters with shared ownership/peer history
- Geometry: reject inconsistent path-loss fittings or impossible timing; robust outlier removal
- Density fairness: apply LDM per epoch; include 3D proximity (e.g., ≤ 1 m horizontal, ≤ 2 m vertical)
- Witness confidence: use DOP and witness distribution; low-confidence windows get lower weight

Parameters (DAO-tunable starting points)

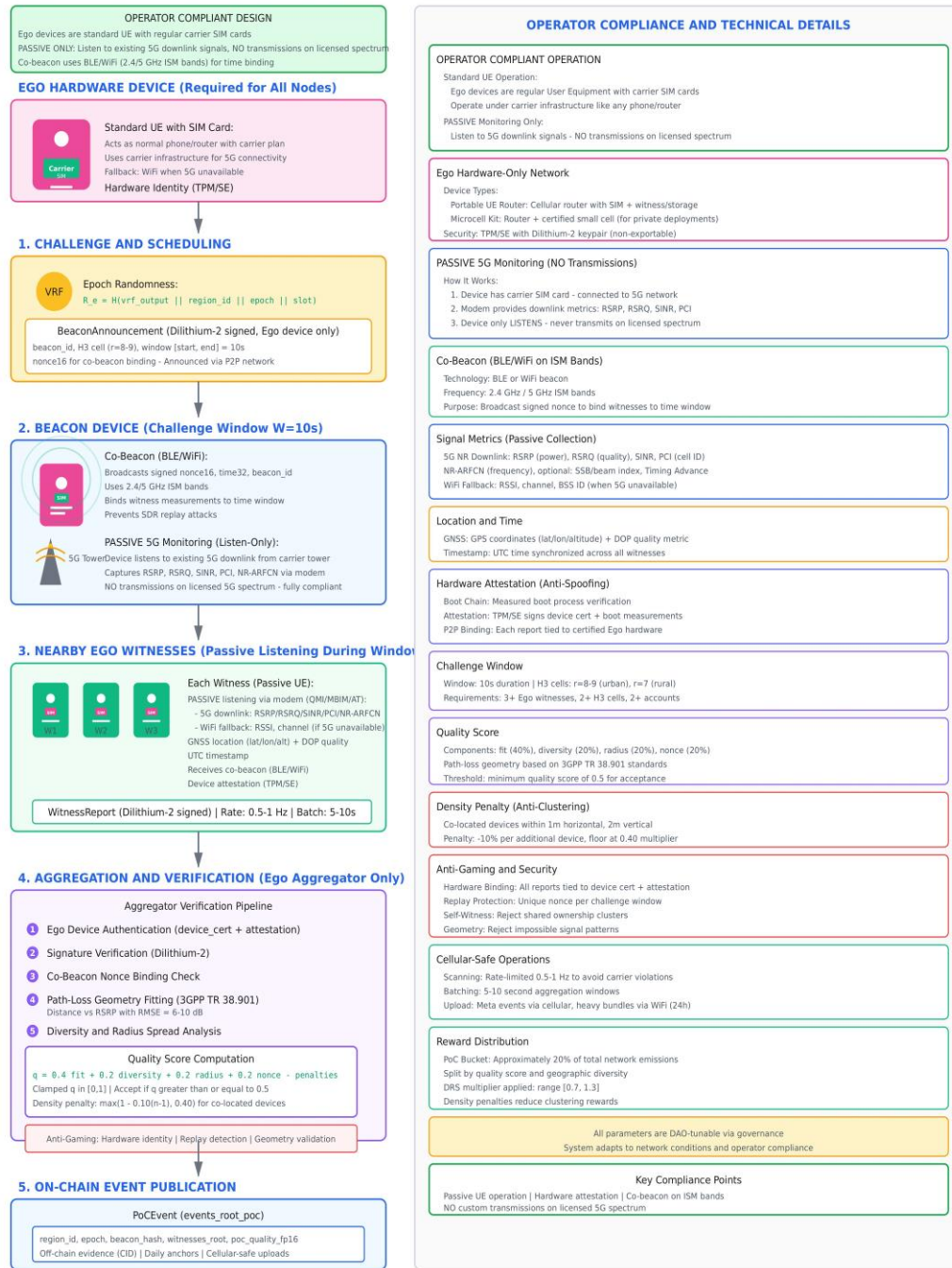
- H3 resolution: $r=8-9$ urban; $r=7$ rural
- Window W: 10 s; announcement 1–2 s before start
- Min witnesses: 3; diversity: ≥ 2 H3 cells and ≥ 2 accounts
- Fit thresholds: RMSE $\tau \approx 6-10$ dB; $q_{\min} = 0.5$
- Density penalty: -10% per extra device, floor at 0.40
- Cellular-safe defaults: scan 0.5–1 Hz; batch 5–10 s; meta events on cellular; bundles uploaded over Wi-Fi within 24 h anchor window

Integration with DRS and rewards

- DRS PoC inputs: accepted q (post-LDM), witness and geometry pass ratio, replay/density penalties
 - Rewards: PoC bucket (e.g., 20% of emissions) split across accepted events by q and diversity, then scaled by DRS multiplier $m \in [0.7, 1.3]$
 - Explorer: shows time-stamped coverage events, q and LDM, and evidence links
 -
 - Proof-of-Replication (PoRep)
 - Sealing: $\text{replica_id} = H(\text{"ego/replica/v1"} \parallel \text{piece_id} \parallel \text{node_pk} \parallel \text{nonce}_{32})$; seal via replica-bound encoding (Stacked-DRG class) to CommR; produce a PoRep proof; PoRepEvent signed with Dilithium-2.
 - Proof-of-Spacetime (PoSt)
- Challenges: per-epoch from `vrf_output`; sample leaves deterministically per sector/replica; prove (WindowPoSt) with optional aggregation; post PoStEvent digests (latency/result), evidence off-chain by CID.

Ego Device Proof-of-Coverage (PoC) Workflow

Operator Compliant - Passive 5G UE Monitoring with Hardware Attestation



Proof of Spacetime (PoSt)

Purpose

- Prove continuous custody and availability of sealed data over time (not just at a spot challenge).
- Back storage payments and slashing with verifiable, auditable proofs.

Pre-requisite: Proof of Replication (PoRep)

- Sealing (Stacked-DRG class): each replica bound to node identity via $\text{replica_id} = H(\text{"ego/replica/v1"} \parallel \text{piece_id} \parallel \text{node_pk} \parallel \text{nonce32})$
- Outputs: CommD (data root), CommR (replica root), PoRep proof; PoRepEvent signed and posted with off-chain evidence CID

Sectoring and schedule

- Sectors: fixed-size sealed units (e.g., 32/64 GiB), containing packed pieces
- Proving windows: each epoch (e.g., 10–30 min) challenges a subset (WindowPoSt); SLA window Δ (e.g., 3–10 min) for submission
- Randomness per epoch: $R_e = H(\text{vrf_output} \parallel \text{deal_id} \parallel \text{epoch})$
- Sampling: choose s leaf indices per sector: $\text{index_i} = \text{PRF}(\text{replica_id}, R_e \parallel i) \bmod N_{\text{leaves}}$

Proof construction and submission

- For each challenged sector/partition:
 - Open Merkle branches for sampled leaves (to CommR) and/or prove WindowPoSt circuit
 - Aggregate proofs (optional SNARK aggregation) to reduce verification cost
- Submit PoStEvent (meta on-chain under events_root_post): $\{\text{deal_id}, \text{node_addr}, \text{replica_id}, \text{epoch}, \text{window_id}, \text{challenges_root}, \text{post_proof_hash}, \text{result}\{\text{pass}\|\text{miss}\|\text{fault}\}, \text{latency_ms}, \text{cid_hint}, \text{node_sig}\}$
- Off-chain: PoStEvidence $\{\text{epoch}, R_e, \text{indices}, \text{openings}/\pi_{\text{post}}, \text{logs}, \text{node_sig}\}$; anchor daily roots

Health, repair, and slashing (RF=3 triads)

- Counters: per-replica pass/miss with rolling window W epochs (e.g., 24)
- Soft fault: small number of misses $\rightarrow$ zero payout that epoch; DRS penalty
- Hard fault: consecutive misses or threshold exceeded $\rightarrow$ slash collateral; promote healthy replica to Primary; repair new replica (backfill from healthy), redo PoRep, and resume PoSt
- Events: RepairEvent and SlashEvent recorded; triad state updated

Parameters (DAO-tunable)

- s (samples/epoch): production 256–1024 depending on sector count; MVP 64–128
- T_{epoch} : 10–30 min; SLA Δ : 3–10 min; health window W : ~ 4 –12 h
- Sector sizes: 32/64 GiB; tree arity (e.g., 8-ary) for lower depth
- Evidence anchoring: daily off-chain roots; auditors can re-verify from anchors

Bandwidth/ops discipline (cellular-safe)

- Meta events (PoStEvent) allowed over cellular
- Heavy evidence bundles buffered and uploaded on Wi-Fi/Ethernet within the 24 h anchor window (unless SLA requires immediate)
- Aggregators can batch-verify and SNARK-aggregate to minimize network load

Integration with DRS and rewards

- DRS PoSt inputs: pass ratio, latency distribution, fault/miss counts within W

- Rewards: Storage bucket (e.g., 55% of emissions) paid per valid PoSt; zero for misses; DRS multiplier $m \in [0.7, 1.3]$ applied; slashing strictly evidence-based
- Retrieval audits (serve ratio) can further influence DRS

Auditing and disputes

- PoC/PoS evidence bundles are content-addressed (CID) and anchored daily; on-chain PoCEvent/PoSEvent carry digest/cid_hint
- Fraud proofs:
 - PoC: InvalidInclusion/InvalidWitnessSet (incoherence or replay) → slash aggregator/operator if proven
 - PoSt: InvalidPoStAggregate or invalid openings against committed CommR → slash provider and invalidate rewards for the window

Node responsibilities (summary)

- Beacon device (PoC): publish challenge announcements; co-beacon nonce over BLE/Wi-Fi; log NR context
- Witness: capture LTE/NR metrics; record co-beacon; batch and sign WitnessReport; respect cellular-safe caps
- Aggregator: verify signatures; run coherence and scoring (fit/diversity/radius/nonce; apply LDM); post PoCEvent; store bundle CID and anchor daily
- Storage provider: seal deals (PoRep); answer PoSt challenges within SLA; emit PoStEvent; upload evidence bundles; handle repair/promotion if faults occur

Why this structure works

- Operator compliance: PoC does not require custom transmissions on licensed NR; BLE/Wi-Fi side-channel nonce binds time/identity legally.
- Deterministic and auditable: deterministic scoring and daily anchors let anyone verify proofs; DRS uses only evidence-backed inputs.
- Robust incentives: PoSt ensures continuous storage; PoC rewards coverage quality and diversity; density penalties discourage clustering; DRS ties everything into transparent rewards and quotas.

EgoStorage: PoRep and PoSt Workflows

GPU-Accelerated | Stacked-DRG | WindowPoSt | RF=3 Triads | Hardware Attested

PROOF-OF-REPLICATION (PoRep)

Purpose: Prove unique, replica-bound encoding of data sealed into sectors

Data Model

Sector Structure:
Sector: Fixed size (32 GiB or 64 GiB) sealed unit
Piece: Client-encrypted content (XChaCha20-Poly1305)
CommP: Merkle root over padded piece
CommD: Unsealed data root (all pieces in sector)

Replica Identity:
 $replica_id = H("ego/replica/v1" || sector_id || node_pk || nonce32)$
Unique per sector per node
Binds to Ego device TPM/SE

PHASE 1: PreCommit1 (PC1) - Packing

CommR: Replica root (final commitment after encoding)
Data Packing (CPU + Disk I/O):



1. Pack multiple pieces into sector
2. Build piece Merkle trees for each piece
3. Compute CommD (unsealed data root)
4. Persist piece alignment map

Duration: 30-45 seconds | Output: CommD and piece_map

PHASE 2: PreCommit2 (PC2) - SDR Encoding

**Stacked-DRG Encoding (GPU-Accelerated):**

Algorithm: Stacked-DRG with L=11 layers, d=6 base degree
1. Generate graph with replica_id-seeded PRF
2. Each layer node derived from parents (cryptographic permutation)
3. Final layer nodes Merkleized to CommR (replica root)
4. Build replica Merkle tree (8-ary for performance)

Duration: 45-60 seconds | Output: CommR
 $PreCommit = CommD, CommR, replica_id, sector_id, params_hash$

PHASE 3: Commit (C1/C2) - Proof Generation

**PoRep SNARK Proof (GPU with MSM/FFT):**

Generate proof: CommR arises from CommD via SDR
Uses replica_id binding under params (Groth16/Plonk)
Proof shows unique encoding without revealing data

Duration: 20-30 seconds | Output: SNARK proof
Succinct: Fast on-chain verification (under 5ms)

PHASE 4: On-Chain Publication

PoRepEvent (On-Chain Metadata)

deal_id, sector_id, node_addr, replica_id, CommD, CommR
porep_params_v, proof_hash, node_sig (Dilithium-2)
Off-chain: PoRepEvidence with proof stored by CID
Regional rollout verifies and posts digest | Daily anchors

PROOF-OF-SPACETIME (PoSt)

Purpose: Prove continuous custody and availability of sealed data over time

Challenge Scheduling

Epoch-Based Challenges:
T_epoch: 20 minutes default | SLA: 8 minutes to submit proof
Proving windows: Each sector appears in windows per day
Partitions: Group sectors (approx 2349 sectors/partition)

$R_e = H(vrf_output || node_addr || epoch || window_id)$

CHALLENGE GENERATION (Per Epoch)

Deterministic Challenge Selection:

For each sector in window, derive s challenged leaves (s=24-48)
 $leaf_index_j = PRF(replica_id, R_e || j) \bmod tree_leaves$
Challenges bind to VRF seed + replica_id
No provider influence - unpredictable and fair

PROOF CONSTRUCTION (WindowPoSt)

**WindowPoSt Proof (GPU-Accelerated):**

1. Open Merkle paths for challenged leaves in each sector CommR
2. WindowPoSt circuit verifies all openings + replica binding
3. Aggregatemultiple partition proofs into single SNARK
4. Target: 1-3 aggregate proofs per epoch per provider

Duration: 15-30 seconds for 100-500 partitions
On-chain verification: Under 3ms per aggregate proof

PoSt EVENT PUBLICATION

PoStEvent (On-Chain)

node_addr, epoch, window_id, partitions_covered, challenges_root
post_agg_proof_hash, result (pass/partial/miss/fault), latency_ms
Off-chain: PoStEvidence with openings and proofs stored by CID

HEALTH MONITORING AND RF=3 TRIADS

Triad Structure: Primary + ReplicaA + ReplicaB

Each piece/deal has 3 independent replicas with unique replica_id
Health per replica: pass_ratio, consecutive_misses, last_seen_epoch

Fault Handling:
Soft Fault (1 miss): Zero payout for window, DRS penalty
Hard Fault (2+ misses): Mark faulty, promote healthy replica to Primary

Repair Process:
1. Select new node | 2. Fetch from healthy replica | 3. Seal with new replica_id

ECONOMICS AND REWARDS

Per-window payout: Base if PoSt accepted, scaled by DRS multiplier [0.7, 1.3]
Slashing: Hard faults slash 1-5% collateral | Invalid proofs terminate deals
Storage bucket: Approximately 55% of emissions by pass_ratio and latency

KEY PARAMETERS AND TECHNICAL DETAILS

Sector Specifications

Sector Sizes: 32 GiB (default), 64 GiB (optional)
Merkle Tree: 8-ary (faster, smaller depth)
Sectors per Partition: Approx 2349 (tunable per system)

Stacked-DRG Parameters

Layers (L): 11 | Base Degree (d): 6
Parent Selection: Deterministic via replica_id-seeded PRF
Params Hash: porep_params_v stored on-chain for versioning
Security: Prevents outsourcing, generation, Sybil attacks

WindowPoSt Parameters

Windows per Day: 48-72 | Each sector appears at least once/day
Challenges per Sector (s): 24-48 per window
Detection: Less than 2 to power -12 per window for missing sectors
Aggregation: Up to 128-512 partitions per aggregate proof

Cryptography (Post-Quantum)

Signatures: ML-DSA-2 (Dilithium-2) for all events and commitments
KEM: ML-KEM-768 (Kyber-768) for content encryption per provider
Hash: BLAKE2s-256 with domain separation
Sessions: X25519+Kyber hybrid for P2P streams

Ego Hardware Requirements

Device Identity: Dilithium keypair in TPM/SE (non-exportable)
Attestation: Device cert + boot measurements required
GPU: Mid-range for sealing/proving acceleration
Storage: NVMe with 1+ GB/s sustained read/write
All storage nodes must be certified Ego hardware

Performance Targets (32 GiB Sector)

PC1 (Packing): 30-45 seconds (CPU + NVMe)
PC2 (SDR Encode): 45-60 seconds (GPU + Tree build)
Commit Proof: 20-30 seconds (GPU SNARK generation)
WindowPoSt: 15-30 seconds for 100-500 partitions
On-Chain Verification: Under 3ms per aggregate

Randomness and Fairness

VRF-Based: Deterministic, verifiable challenge selection
Anti-Grinding: Challenges bind to finalized header VRF output
No Provider Influence: Sector assignment to windows deterministic

RF=3 Triad System

Structure: Primary + ReplicaA + ReplicaB for each deal/piece
Each replica sealed separately with unique replica_id
Independent failure domains ensure data availability
Repair timeout: 24 hours | Escalate slashing after 72 hours

Fault Detection and Thresholds

Soft Fault: 1 miss in window - Zero payout, DRS penalty
Hard Fault: 2+ misses in rolling window - Mark faulty, slash collateral
Promotion: Healthy replica elevated to Primary automatically
Repair: Fetch from healthy, seal with new replica_id, resume PoSt

On-Chain vs Off-Chain Data

On-Chain: Event metadata, digests, proof hashes, commitments
Off-Chain: Full proofs, evidence bundles stored by CID
Anchoring: Daily EvidenceRoot commits to off-chain data
Cellular-Safe: Meta via cellular, heavy uploads via WiFi (24h window)

Dispute Protocol (Fraud Proofs)

Challenge Window: 24-72 hours to dispute any PoRep/PoSt aggregate
InvalidSeal: Prove SDR relation fails under committed params
InvalidPoStAggregate: Prove opening fails against CommR
On Success: Slash provider and rollout collateral, void payouts

Economics and Rewards

Storage Bucket: Approximately 55% of total network emissions
Payout: Base amount per window if PoSt accepted
DRS Multiplier: Range [0.7, 1.3] based on pass_ratio and latency
Slashing: 1-5% collateral for hard faults, deal termination for fraud

Sector Lifecycle

Sealing queued → PC1 → PC2 → Commit → Proving (Active)
Active: Sectors participate in scheduled windows, must submit PoSt
Extend/Renew: Extend sector lifetime, no re-seal required
Terminate/Expire: Leave proving set, retention window, secure erase

Security and Anti-Cheating

Anti-Outsourcing: replica_id-bound SDR, changing media requires re-seal
Anti-Grinding: Challenges bound to VRF seeds, no provider influence
Equivocation: Duplicate sectors detected via state, slashing on detection
Availability: Missing evidence upload triggers penalties
Confidentiality: Providers store ciphertext only, keys via Kyber KEM

Storage Provider Node Architecture

Sealing Manager: Queue, GPU workers, CommD/CommR persistence
Prover: WindowPoSt scheduler, GPU proof generation, aggregation
Storage Index: Sector metadata, proving partitions, health counters
Publisher: Emit events immediately, queue evidence for WiFi upload
Security: TPM/SE Dilithium signing, attestation at P2P Identify

Regional Rollup Responsibilities

Collect PoRep/PoSt events and evidence from providers
Batch-verify with GPU/SIMD acceleration
Post on-chain digests and publish evidence bundles by CID
Anchor daily EvidenceRoot, accept and process fraud proofs

Testing and Verification

Determinism: Independent implementations produce identical challenges
Soundness: Simulate sector removal, measure detection probability
End-to-End: Seal → prove → induce faults → repair → slashing
Audits: Recompute payouts from on-chain roots and off-chain bundles

All parameters are DAO-tunable via governance
System adapts to storage needs and hardware capabilities

Production-Grade Storage Proofs

GPU-accelerated | Hardware-attested | Fraud-proof secure | Operator-compliant

Deterministic Reward Scoring (DRS)

- - Purpose: transparent, reproducible scoring that drives reward multipliers and quotas.
- - Inputs (per node/epoch): uptime ratio, PoSt pass rate and latency, PoC quality q (already includes density penalty), serve ratio, bandwidth/resource honesty; penalties for proven fraud/equivocation.
- - Output: a score in $[0,1]$ and multiplier $m \in [0.7, 1.3]$, applied per reward bucket. All inputs are evidence-backed from on-chain events and verifiable bundles.

How DRS uses PoC/PoS

Purpose

- Convert verifiable network behavior into a transparent, reproducible score and a bounded reward multiplier that is applied to each node per epoch.
- Inputs come only from evidence-backed, on-chain anchored events and their off-chain bundles (auditable by anyone).

Data flow (per epoch)

Evidence production

- PoC: Aggregators emit PoCEvent {region_id, epoch, beacon_hash, witnesses_root, poc_quality_fp16, cid_hint, ...}, with witness bundles (CBOR/Protobuf) and a daily evidence anchor root.
- PoSt: Providers emit PoStEvent {deal_id/node_addr, epoch, challenges_root, post_proof_hash, result, latency_ms, cid_hint, ...}, with PoStEvidence and daily anchors.
- Uptime/serve: Uptime derived from consensus participation and health; serve_ratio from retrieval receipts and audit downloads.

Ingestion and counters

- For each node/account, Ego core accumulates per-epoch counters:
 - uptime_slots_seen, uptime_slots_expected
 - post_challenges, post_passes, post_latency_sum_ms, post_latency_count
 - poc_events_accepted, poc_q_sum (already post-density LDM), witness_confidence_sum
 - serve_bytes_ok, serve_bytes_requested
 - penalties (counts/flags): replay/incoherence, invalid proofs, equivocation

Normalization (all metrics in $[0,1]$)

- $uptime = \min(uptime_slots_seen / \max(1, uptime_slots_expected), 1)$
- $post_pass = \min(post_passes / \max(1, post_challenges), 1)$
- $avg_post_latency_ms = post_latency_sum_ms / \max(1, post_latency_count)$
- $post_latency_norm = \text{clamp}(avg_post_latency_ms / SLA_ms, 0, 1)$
- $inv_latency = 1 - post_latency_norm$
- poc_quality:
 - $q_avg = (\sum q_i \cdot w_i) / \max(\epsilon, \sum w_i)$, where q_i is the per-event quality after LDM, and w_i = witness_confidence or 1
 - $poc_quality = \text{clamp}(q_avg, 0, 1)$
- $serve_ratio = \min(serve_bytes_ok / \max(1, serve_bytes_requested), 1)$
- penalties term p :
 - $p = \text{clamp}(a1 \cdot failed_post + a2 \cdot replay_or_incoherence + a3 \cdot equivocation, 0, p_max)$
 - Use small, DAO-tunable weights (e.g., $p_max \leq 0.5$) so penalties cannot drive negative scores

Scoring (deterministic, versioned)

- Parameters (weights_version):
 - Weights $w_uptime, w_post_pass, w_inv_latency, w_poc, w_serve$ (sum ≤ 1)
 - Penalty weights $a1, a2, a3$; smoothing $\alpha \in [0,1]$; multiplier slope β ; bounds

- $m_{\min}, m_{\max}$
 - SLA_ms for PoSt; confidence rules for witnesses
 - Raw score:
 - $$\text{score_raw} = \frac{w_{\text{uptime}} \cdot \text{uptime}}{w_{\text{inv_latency}} \cdot \text{inv_latency} + w_{\text{poc}} \cdot \text{poc_quality} + w_{\text{serve}} \cdot \text{serve_ratio} - p} +$$
 - Smoothing (optional):
 - $\text{score} = (1 - \alpha) \cdot \text{score_prev} + \alpha \cdot \text{score_raw}$
 - Clamp final score to [0,1]

Multiplier and bands

- Baseline-centered multiplier:
 - $m = 1 + \beta \cdot (\text{score} - 0.5)$, clamped to $[m_{\min}, m_{\max}]$, e.g., [0.7, 1.3]
- Apply m separately to each reward bucket (Storage/Consensus/Coverage).
- Quota bands (optional):
 - High (>0.8): relaxed quotas and fewer audits
 - Mid (0.5–0.8): standard quotas
 - Low (<0.5): tighter quotas, extra audits, lower publish rates on heavy topics

Reward mapping (per epoch)

- Emission per bucket E_{bucket} (DAO-tunable) and per-node base share i :
 - Storage: byte-months under deal $\times$ participation in the epoch's PoSt window (0 if miss)
 - Coverage: sum of accepted PoC q (post-LDM) for the epoch in the node's region(s)
 - Consensus: stake participation and proposer/vote share (optionally with PUC)
- Weighted share:
 - $w_{\text{share}}_i = \text{base_share}_i \times m_i$
- Payout:
 - $\text{payout}_i = E_{\text{bucket}} \times w_{\text{share}}_i / \sum_j w_{\text{share}}_j$
- Missed/invalid events yield $\text{base_share}_i = 0$ for the epoch; slashing is evidence-based and handled separately

Anchoring and transparency

- At epoch boundary, core emits:
 - DRSScoreEvent {node_id, epoch, score_u32, multiplier_fp16, evidence_root[32], weights_version, params_digest, sig}
- Evidence roots (daily) commit to PoC/PoSt bundles; auditors recompute metrics and DRS to verify the published score and multipliers.

Example (starting parameters; DAO-tunable)

- Weights: $w_{\text{uptime}}=0.20$, $w_{\text{post_pass}}=0.40$, $w_{\text{inv_latency}}=0.10$, $w_{\text{poc}}=0.20$, $w_{\text{serve}}=0.10$
- Penalties: $a_1=0.10$ (failed_post), $a_2=0.20$ (replay/incoherence), $a_3=0.40$ (equivocation); $p_{\max}=0.5$
- SLA_ms (PoSt): 600,000 (10 minutes)
- Smoothing $\alpha=0.3$; multiplier slope $\beta=0.6$; bounds $[m_{\min}, m_{\max}]=[0.7, 1.3]$

How PoC feeds DRS (details)

- q_i per event includes:
 - Geometry fit (38.901), diversity (distinct H3 cells/accounts), radius spread, nonce-binding fraction
 - Density penalty LDM (−10% per extra device in ~1 m², floored at 0.40)
- Witness weighting:
 - w_i = confidence from GNSS DOP, witness distribution, timing advance consistency; cap per-cell witness contribution to avoid clustering
- Aggregation:

- `q_avg` is computed across accepted events for the node in the epoch; `PocEvent` provides `poc_quality_fp16` and `witnesses_root`; evidence bundle includes witness set and stats used

How PoSt feeds DRS (details)

- pass ratio = passes/challenges over scheduled windows in the epoch
- latency:
 - `avg_post_latency_ms` vs `SLA_ms` yields `inv_latency = 1 - clamp(avg/SLA, 0, 1)`
 - Outliers handled by winsorization or truncation to avoid single-latency spikes dominating
- Consecutive misses:
 - Miss counters in a rolling window `W` are tracked separately for health and slashing; DRS penalties apply only to the epoch's score (e.g., `a1 * failed_post`)

Quotas and anti-spam coupling

- DRS bands adjust RU quotas, publish rates, and audit frequency:
 - High band: higher `ru_limit` caps, higher proof batch sizes
 - Low band: lower `ru_limit` caps, reduced batch sizes, more frequent retrieval audits
- Optional PoB floors raise admission priority only when enabled by DAO; they do not alter DRS directly

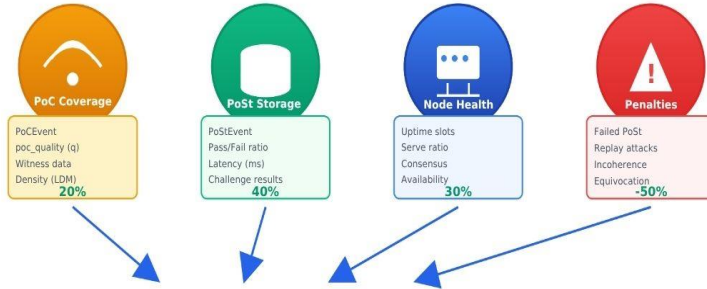
Pseudocode (simplified)

- `inputs = { uptime, post_pass, avg_post_latency_ms, poc_q_events[], serve_ratio, failed_post_count, replay_or_incoherence_count, equivocation_count }`
- `inv_latency = clamp(1 - avg_post_latency_ms / SLA_ms, 0, 1)`
- `q_avg = weighted_mean([e.q_after_LDM for e in poc_q_events], weights=[e.witness_conf])`
- `p = clamp(a1*failed_post_count + a2*replay_or_incoherence_count + a3*equivocation_count, 0, p_max)`
- `score_raw = w_uptime*uptime + w_post_pass*post_pass + w_inv_latency*inv_latency + w_pocq_avg + w_serve*serve_ratio - p`
- `score = clamp((1 - alpha)*score_prev + alpha*score_raw, 0, 1)`
- `m = clamp(1 + beta*(score - 0.5), m_min, m_max)`
- `emit DRSScoreEvent(node, epoch, score_u32, m_fp16, evidence_root, weights_version, params_digest, sig)`
- Auditing
- Deterministic recomputation:
 - Given the same event stream (`PoCEvent`, `PoStEvent`) and evidence bundles, two independent implementations must compute identical DRS scores and multipliers for a specified `weights_version`.
- Replay protection and versioning:
 - `DRSScoreEvent` includes `weights_version` and `params_digest`; parameter upgrades are time-locked and versioned on-chain.

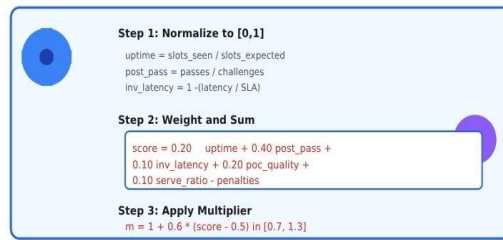
Deterministic Reward Scoring (DRS) System

Evidence-Based Performance Scoring - Multiplier Range: 0.7 to 1.3

EVIDENCE SOURCES (Per Epoch)



NORMALIZATION AND SCORING ENGINE



OUTPUT: DRS SCORE AND MULTIPLIER



REWARD DISTRIBUTION (PerEpoch)

Multiplier Applied to All Buckets

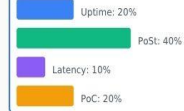


Final Payout Calculation

$$payout = E_bucket \cdot (base_share \cdot multiplier) / \sum(weighted_shares)$$

KEY PARAMETERS

Weight Distribution



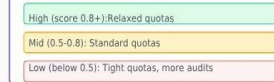
SLA Parameters

PoSt Latency SLA: 600,000 ms (10 min)
Smoothing alpha: 0.3 (30%new, 70% old)

Penalty Weights

Failed PoSt: a1 = 0.10
Replay/Incoherence: a2 = 0.20
Equivocation: a3 = 0.40
Maximum penalty: p_max = 0.5

Performance Bands



Evidence Anchoring

Daily evidence roots commit to bundles
Auditors can recompute all scores
Deterministic and reproducible

Transparency Features

All inputs evidence-backed
No self-reported metrics
On-chain verifiable
Independent audit possible

Example Calculation

Inputs: 95% uptime, 98% PoSt pass
92% latency, 87% PoC, 90% serve
Penalty: 0.05
 $score = 0.19 + 0.39 + 0.09 + 0.17 + 0.09 - 0.05$
 $= 0.88$
Multiplier: 1.23

DAOGovernance

All parameters tunable via governance
Time-locked upgrades for predictability

System Benefits

- Transparent scoring methodology
- Reproducible by independent auditors
- Fair bounded multipliers
- Evidence-backed, no gaming
- Adaptive to network conditions
- Incentivizes quality performance

5. Transactions and execution (feeless UX with metering)

- Resource metering (no fees):
 - `ru_limit`: hard cap on Resource Units (compute/memory/IO/storage writes)
 - `ru_estimate`: sender's estimate (must be $\leq$ `ru_limit`)
 - `pob_burn_credits` (optional): Proof-of-Burn credits (e.g., Deploy Credits) for anti-spam/deploy policy when enabled by DAO
- Optional privacy (stealth outputs):
 - `TxOutStealth` = { `pk_lt`, `enc` { `kyber_ct`, `nonce24`, `aead_ct` } }, where `kyber_ct` is ML-KEM-768 ciphertext and `aead_ct` is XChaCha20-Poly1305 over output metadata (amount/memo); keys derived via HKDF-BLAKE2s from the Kyber shared secret.
- Cross-shard messaging (first-class receipts):
 - `CrossShardReceipt` { `src_shard`, `dst_shard`, `src_block_hash`, `tx_id`, `payload`, `nonce`, `deadline_epoch`, `merkle_proof` }.
 - Destination shard verifies source header+QC and Merkle proof against `src_receipts_root` before applying; enforces nonce/deadline.

Rollups and data availability (DA)

- `RollupCommit` { `rollup_id`, `epoch/window`, `tx_root`, `proofs_root`, `da_root`, `counts`, `operator_sig` } posted to a shard.
- DA blobs: erasure-coded ($k+m$), chunked; `da_root` commits manifest; light clients sample random chunks to detect unavailability.
- Disputes: `FraudProof` types (InvalidInclusion/Transition, DAUnavailability); valid challenges slash operator and invalidate the commit.

Storage, sharding, and archival

- Sharded L1: prefix mapping to shards; cross-shard receipts; per-shard BFT; epoch aggregation for global finality.
- Storage network: RF=3 triads (Primary + two replicas); promotion/repair on faults; erasure coding for large objects; retrieval audits feed DRS `serve_ratio`.
- Archival under PoSt: old block bodies, contract code blobs, state snapshots, rollup blobs, and proof bundles stored off-chain; headers commit roots; auditors verify via on-chain anchors.

Cryptography and addressing (PQC-first)

- Signatures: ML-DSA-2 (Dilithium-2) for blocks/tx/proofs/rollups/governance; optional SLH-DSA (SPHINCS+) for long-term anchors.
- KEM: ML-KEM-768 (Kyber-768) for stealth outputs and stream handshakes.
- Symmetric: XChaCha20-Poly1305; Hash/KDF: BLAKE2s-256 and HKDF-BLAKE2s; domain-separated digests (e.g., "ego/tx/v1", "ego/core/v1").
- Addressing: `address` = `versioned_blake2s(dilithium_pk)[0..20]` (Bech32m HRP `ego/egot/egod`). Users send to short addresses; large keys remain in account state.

Proof-of-Burn (PoB) and deploy/storage credits

- Burn EGO to mint Storage Credits (byte-months) and Deploy Credits (per KB/RU).
- Adaptive PoB floors (per account/epoch) can be enabled by DAO for anti-spam/priority; combined with quotas and DRS bands to preserve feeless UX.

Ledger and ordering

- Ordering/finality come from sharded BFT (not a separate "proof-of-serialization"). `Receipts_root` commits cross-shard messages; deterministic replay ensures all nodes see the same order.

Application layer (contracts and tooling)

- Urego deterministic runtime: compile-to-WASM (deterministic subset), fuel-metered (RU), with host APIs (storage/receipts/events/crypto/PoB/staking).
- Developer UX: DeployPolicy (free staker quota + PoB Credits + bond/caps/dedup); on-chain pointers for safe upgradability (timelocks + events).
- Explorer/light client: client-verifies headers/QCs (Dilithium), state proofs, and DA sampling; no trusted RPCs required.

6. Tokenomics and Staking Rewards

Asset and units

- Coin: EgoCoin (EGOC); smallest unit: uEGOC = 10^{-8}
- Address: versioned blake2s(Dilithium_pubkey)[0..20] (short Bech32m)

Supply and allocations (max 100,000,000 EGOC)

- Emissions to nodes (block rewards): 40,000,000
- Liquidity/Treasury: 20,000,000
- Investors: 20,000,000
- Team: 10,000,000 (vesting/locks via DAO)
- Marketing: 10,000,000

Emission schedule and reward split

- Emission clock: 0.1 s blocks; halving every 2 years (~630,720,000 blocks)
- Initial per-block reward $R_0 \approx 0.03172$ EGOC/block (to emit 40M over infinite halvings)
- Per-block/epoch split (DAO-tunable): Storage (PoSt/PoRep) 55%, Consensus/Proposer 25%, Coverage (PoC) 20%
- Deterministic Reward Scoring (DRS): per-node multiplier $m \in [0.7, 1.3]$ based on uptime, PoSt pass/latency, coverage quality, and serve ratio; fully evidence-backed and auditable
- Adaptive tilt (optional): reallocate up to $\pm 5\%$ per epoch (max $\pm 20\%$) between Storage and Coverage based on scarcity

Feeless UX and anti-spam

- No transaction fees and no deploy fees at the UX level
- Protocol metering only: transactions carry RU limits ($ru_limit/ru_estimate$)
- DeployPolicy: free staker quota (e.g., 5/epoch) + optional PoB Deploy Credits and small deploy bond; hard caps/dedup for abuse prevention
- Optional PoB floors (DAO-controlled) for anti-spam/priority during load

Staking and locks

- New block rewards are locked 30 days and earn 20% simple interest over the lock ($\approx 0.0548\%$ /day on the rolling locked tranche)
- Baseline APRs (DAO-tunable): Node staking $\sim 40\%$; General staking $\sim 20\%$
- Slashing is strictly evidence-based (storage/coverage failures, equivocation)

Market tax (economic layer; transfers remain fee-free)

- 1% buy/sell tax on AMM/CEX trades; wallet-to-wallet transfers are fee-free
- Split α to Staking Rewards Pool and $(1-\alpha)$ to Treasury/Liquidity (e.g., $\alpha = 0.5$; DAO-tunable)

Storage Credits and demand

- Users can burn EGO to mint Storage Credits (byte-months) and pay for storage/retrieval; creates demand-backed revenue for operators alongside emissions

7. Block Details and Transaction Processing

Our blocks are compact and verifiable. Each block header includes a timestamp, previous_hash, and BLAKE2s-committed roots: tx_root, state_root, receipts_root, events_root_post (storage proofs), events_root_poc (coverage proofs), rollup_root, and da_root. Transactions are Merkleized; the proposer signs the BlockHeaderCore with Dilithium-2, and validators finalize via a sharded BFT QC (bitmap + sigs_root). Large data (code blobs, old bodies, DA blobs, proof bundles) stays off-chain and is referenced by these roots for auditability.

Performance comes from sub-second micro-slots (~0.1 s) and pipelined finality (1–3 s global), plus rollups and data-availability. This delivers ≥100k TPS across shards with a feeless UX (transactions carry RU limits for metering, not fees).

How a transaction is processed

- Alice signs a transfer to Bob (Dilithium-2) with ru_limit/ru_estimate and broadcasts it.
- The shard mempool admits the tx by policy (RU/quota/DRS bands); the proposer assembles a candidate block and computes roots.
- Validators verify signatures, state transitions, and roots, then vote; upon 2f+1 votes (QC), the block finalizes and Alice's tx is confirmed within seconds.
- PoC/PoS are recorded as events (not consensus): storage/coverage proofs are posted as digests in events_root_post/poc, with full evidence off-chain and daily anchors for public audit.

8. Block Reward Calculation

We use a fixed emission schedule with 2-year halvings and no fee dependence. A total of $D = 40,000,000$ EGO is emitted via block rewards with a 0.1 s block interval; each halving epoch spans $E = 630,720,000$ blocks (~2 years). Let $k = \text{floor}(\text{years_since_launch} / 2)$ be the halving index and $R_0 = D / (2 \cdot E) \approx 0.03172$ EGO/block the initial per-block reward; the per-block base reward at epoch k is $R(k) = R_0 / 2^k$.

Each block's $R(k)$ is split across reward buckets (Storage 55%, Consensus/Proposer 25%, Coverage 20%). For bucket b with split s_b , node i 's payout is $p_{i,b} = R(k) \cdot s_b \cdot (m_i \cdot w_i) / \sum_j (m_j \cdot w_j)$, where w_i is the node's base share in that bucket (e.g., Storage: byte-months proven in the epoch; Consensus: stake/participation; Coverage: sum of accepted coverage quality) and $m_i \in [0.7, 1.3]$ is the Deterministic Reward Scoring multiplier derived from uptime, storage proof pass/latency, coverage quality, and serve ratio.

9. Nodes

Mainnet nodes are Ego 5G devices (Ego-only). A buyer scans the device serial/QR in the Ego app (mobile/desktop), the app registers the device certificate and TPM/SE attestation, inserts a SIM (or connects Wi-Fi/Ethernet), and the device auto-joins the network. Devices run hardware-backed keys (Dilithium/Kyber), measured/verified boot, and identify on P2P with their device cert; protected topics require valid attestation. For development and early pilots, software nodes on PCs/Orange Pi can be used; mainnet admission remains Ego devices only.

What a device does (roles are composable)

- Storage/PoS: store sealed data, answer randomized PoSt audits within SLA.
- Coverage/Witness: passively measure LTE/NR and co-beacon nonce; submit signed witness reports for coverage events.
- Gateway/Relay/Indexer (optional): serve APIs, relay traffic, index headers/receipts/proofs.
- Validator (SKU/allowlisted): participate in sharded BFT, propose/validate blocks.

Security and anti-abuse

- Consensus safety: sharded BFT (HotStuff/Tendermint-style) with $f < n/3$ Byzantine tolerance per shard; invalid blocks cannot finalize without $2f+1$ Dilithium-signed votes.
- Sybil resistance: one certified device per identity (TPM/SE keys + attestation), P2P Identify binding, optional staking/collateral, DRS-based quotas. Peer scoring, rate limits, and targeted bans mitigate spam/DoS.
- Coverage integrity: multi-witness, UE-compliant PoC (RSRP/RSRQ/SINR/PCI/ARFCN + BLE/Wi-Fi nonce) with 3GPP-based geometry checks, replay protection, and diversity requirements; density penalty applies -10% per extra device co-located in $\sim 1 \text{ m}^2$ (time-weighted), floored at 0.40 LDM.
- Storage integrity: PoRep binds replicas to device identity; PoSt audits enforce continuous custody; promotion/repair restores $RF=3$ on faults; slashing is evidence-based.
- Placement fairness: triads must span distinct H3 cells/regions to avoid co-location; cellular-safe defaults limit heavy uploads to Wi-Fi/Ethernet.

Operations and reliability

- Built in Erlang/OTP for supervision, crash isolation, and large-scale concurrency; libp2p mesh with backpressure and per-topic caps.
- Deterministic Reward Scoring (DRS) converts evidence (uptime, PoSt pass/latency, coverage quality, serve ratio) into a bounded multiplier for rewards and quotas; repeated faults/slashing lower rewards and limit publish rates.
- Joining is zero-touch for users: scan, connect, and earn. Devices can be managed from the app (health, proofs, rewards), with logs/evidence anchored for audit.

10. DAO

Decentralized Autonomous Organization, is a concept that allows for decentralized decision making and control. In Ego Blockchain, we will use a DAO to split the profits from selling the Ego 5G devices. This ensures that the decision-making power is distributed among the members of the community, and no single entity or individual has undue control over the network. We will operate truly decentralized by solving problems through DAO votings. The community members will be able to vote on important decisions related to the development and growth of the network, such as changes to the protocol, the allocation of resources, and the development of new features. In addition, we will use smart contracts to choose the tender winner of doing a task. This ensures transparency and fairness in the selection process, as the winner will be determined based on pre-defined criteria encoded in the smart contract. Every profit and sale from the hardware will be transparently recorded and made available on our website. This ensures that the community members are aware of the financial activities of the network, and are aware how the profits are distributed. In order to ensure fair and transparent decision-making within our ecosystem, we will implement two types of voting and the results it will be averaged to determine the final outcome. The first type of voting requires staking, which means that participants must hold a certain amount of EGOCOIN in order to vote on proposals. The second type of voting is based on passing a knowledge test about the particular subject being voted on. This allows participants to prove their expertise in the area and ensures that only those with a good understanding of the topic are able to vote on it. This will help to prevent uninformed decisions that could negatively impact our ecosystem.

Once both types of voting have taken place, we will calculate an average between the two results to determine the final outcome of the vote. This provides that both stakers and knowledgeable participants have an equal say in the decision-making process, and helps to assure that decisions are made in the best interest of our ecosystem.

Here's an example of how the voting process will work with the two different DAO votings:

►Staking voting: In this type of voting, individuals would stake a certain amount of EGOCOINS to participate in the decision-making process.

The amount staked would be proportional to the voting power of each individual, meaning that those who stake more would have a greater say in the outcome of the vote. The formula for staking-based voting will be:

$$\text{Total Voting Power} = \Sigma (\text{Individual Stake} / \text{Total Staked})$$

For example, if there were 100 people participating in the vote and the total amount staked was 10,000 tokens, and an individual staked 500 tokens, their voting power would be:

$$(500/10,000) \times 100 = 5\% \text{ of the total voting power}$$

►Knowledge-based voting: In this type of voting, individuals would need to pass a test to demonstrate their understanding of the particular subject that is being voted on. Once they pass the test, they would be granted a certain amount of voting power. The formula for knowledge-based voting will be:

$$\text{Voting Power} = (\text{Test Score} / \text{Max Test Score}) \times \text{Base Voting Power}$$

For example, if the base voting power was 10 and an individual scored 80% on the knowledge test with a maximum possible score of 100, their voting power would be:

$$(80/100) \times 10 = 8$$

►Combining the results: To get the final outcome of the vote, we will take an average of the results from both the staking and knowledge-based voting systems.

For example, if the staking-based voting gave a total voting power of 60% and the knowledge-based voting gave a total voting power of 40%, the final voting power will be:

$$\text{Final Voting Power} = (\text{Staking Voting Power} + \text{Knowledge Voting Power}) / 2$$

$$\text{Final Voting Power} = (60 + 40) / 2 = 50$$

In this example, the final voting power is 50%, meaning that the decision would be made based on the majority vote.

11. Conclusion

Our goal is to create a truly decentralized ecosystem that provides secure and reliable services to users. One of our primary objectives is to provide decentralized cloud storage that is both secure and cost-effective. With our blockchain technology, users will be able to store and access their data without worrying about security breaches or data loss. We will also provide a decentralized website that is secure, fast, and reliable.

Another key feature of our ecosystem is offline transactions and offline data storage. We will create a system that allows users to make transactions and store data even when they are offline. This is particularly useful in areas with poor internet connectivity or during natural disasters when internet access may be limited.

We are committed to provide a fee-less environment for our users. We will not charge any transaction fees for sending or receiving egocoins. Additionally, there are also no fees associated with deploying a smart contract on our platform.

Our staking rates will start at 20% APY, which is higher than many other staking platforms. This will incentivize users to hold our tokens and stake them on our platform.

Our blockchain infrastructure is designed to be easy to use for both developers and non-developers. We believe that blockchain technology should be accessible to everyone, and we have designed our ecosystem with this principle in mind.

The EGO 5G device will be created to allow for 5G internet sharing, enabling users to earn passive income by sharing their excess bandwidth with others. This will create a truly decentralized network of wireless networks, providing a more reliable and cost-effective alternative to traditional internet service providers.

With our AI technology, everyone will be able to write smart contracts and create dApps without needing to have any coding experience. This will democratize the blockchain space and allow for more people to participate in the development of decentralized applications.

Finally, we believe that anyone should be able to generate passive income with EGO 5G devices, which store data and provide wireless networks. By leveraging our blockchain technology, we aim to provide a more decentralized and equitable distribution of wealth.

References

- [1] Transition to Post-Quantum Cryptography Standards
<https://nvlpubs.nist.gov/nistpubs/ir/2024/NIST.IR.8547.ipd.pdf>
- [2] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008.
- [3] Juan Benet. IPFS - Content Addressed, Versioned, P2P File System. 2014.
- [4] Protocol Labs. Technical Report: Proof-of-Replication. 2017.
- [5] The Qogecoin Authors. qogecoin, 2021. <https://github.com/qogecoin/qogecoin>, accessed May 10, 2022.
- [6] Vitalik Buterin. Ethereum, 2014
- [7] Tal Moran, Ilan Orlov, " Simple Proofs of Space-Time and Rational Proofs of Storage " pages 4-23.
- [8] P. Labs. Filecoin: A decentralized storage network, 2017. URL: <https://filecoin.io/filecoin.pdf>.
- [9] Helium. Proof of Coverage. <https://docs.helium.com/blockchain/proof-of-coverage/> 2013
- [10] Jonathan Rodriguez. Fundamentals of 5G Mobile Networks:
"https://pce-fet.com/common/library/books/50/6998_[Jonathan_Rodriguez]_Fundamentals_of_5G_Mobile_Net(b-ok.org).pdf" 2015.
- [11] Phillip Rogaway. "Nonce-Based SymmetricEncryption".
<https://www.cs.ucdavis.edu/~rogaway/papers/nonce.pdf> pages 4-11.